TecPro

TECNOLOGIAS

Desenvolvimento, Apostilas e Cursos Online



Fluxograma

se...entao

Portugol

C++

Linguagem



Repetição

Algoritmos e Programação

Lógica, Portugol e C++

Algoritmos e Programação

Lógica, Portugol e C++

TecPro Tecnologias

Desenvolvimento, Apostilas e Cursos Online

2026

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada ou transmitida de qualquer forma, ou por qualquer meio, sem autorização prévia da TecPro Tecnologias.

Sumário

Capítulo 1 – Lógica, algoritmos e o computador **4**
1.1 Lógica . . . 4
1.2 Algoritmos . . . 4
1.3 Como o computador executa um algoritmo . . . 5
1.4 Do problema ao algoritmo . . . 7
1.5 Fluxogramas . . . 7
1.6 Portugol . . . 8
Teste de mesa . . . 9
1.7 Do Portugol ao C++ . . . 10
Erros Comuns . . . 11

Capítulo 2 – Linguagens de programação **12**
2.1 O que é uma linguagem de programação . . . 12
2.2 Sintaxe e semântica . . . 12
As peças de um programa . . . 13
2.3 Compilação e interpretação . . . 13
As etapas da compilação em C++ . . . 14
2.4 O ambiente de programação . . . 15
2.5 A estrutura de um programa em C++ . . . 15
2.6 Saída de dados com cout . . . 16
2.7 Entrada de dados com cin . . . 17
2.8 Um programa completo . . . 17
2.9 Lendo as mensagens de erro . . . 18
Erros Comuns . . . 19

Capítulo 3 – Variáveis, tipos e operadores **20**
3.1 Variáveis . . . 20
3.2 Tipos de dados . . . 21
3.3 Operadores aritméticos . . . 22
3.4 Conversões de tipo . . . 23
3.5 Operadores relacionais . . . 24
3.6 Operadores lógicos . . . 24
3.7 Precedência geral . . . 25
3.8 Funções matemáticas . . . 26
3.9 Um programa completo . . . 26
Erros Comuns . . . 27

Capítulo 1 – Lógica, algoritmos e o computador

Programar é, antes de tudo, organizar o pensamento. Antes de escrever uma única linha em uma linguagem de programação, é preciso entender o problema, decidir quais passos o resolvem e em que ordem executá-los. Neste capítulo você vai conhecer as ideias que sustentam todo o resto da disciplina: o que é lógica, o que é um algoritmo, como um computador executa instruções e duas formas de escrever algoritmos antes de programar, o **fluxograma** e o **Portugol**. No final, vai ver o mesmo algoritmo escrito em C++, a linguagem que usaremos ao longo da apostila.

1.1 Lógica

Lógica é o estudo das formas corretas de raciocinar: como chegar a conclusões verdadeiras a partir de informações verdadeiras. Você a usa o tempo todo sem perceber. Se o aplicativo diz que vai chover e você vai sair, então leva o guarda-chuva. Se o saldo da conta é menor que o preço, então a compra não passa.

O ponto de partida da lógica é a **proposição**: uma frase que pode ser classificada como verdadeira ou falsa, mas não as duas coisas ao mesmo tempo. "A nota é maior ou igual a 6" é uma proposição; "Estude para a prova!" não é, porque não faz sentido perguntar se é verdadeira.

Proposições podem ser combinadas com três conectivos, que vão aparecer em todos os programas que você escrever:

- **E**: "tem nota maior ou igual a 6 **e** frequência maior ou igual a 75%" só é verdadeira quando as duas partes são verdadeiras.
- **OU**: "paga meia entrada se é estudante **ou** tem mais de 60 anos" é verdadeira quando pelo menos uma das partes é verdadeira.
- **NÃO**: inverte o valor. Se "a porta está aberta" é verdadeira, "a porta **não** está aberta" é falsa.

A **tabela-verdade** mostra o resultado para todas as combinações possíveis. Nela, V significa verdadeiro e F, falso:

A	B	A E B	A OU B	NÃO A
V	V	V	V	F
V	F	F	V	F
F	V	F	V	V
F	F	F	F	V

Na programação, a lógica é indispensável porque o computador faz **exatamente** o que mandamos, nem mais nem menos. Ele não adivinha intenções nem corrige um raciocínio mal construído. Se a condição de aprovação for escrita com OU no lugar de E, um aluno com nota 9 e frequência de 30% será aprovado, e o computador não verá problema nenhum nisso.

1.2 Algoritmos

Um **algoritmo** é uma sequência finita de passos bem definidos que resolve um problema. A palavra vem do nome do matemático persa Muhammad ibn Musa al-

Khwarizmi, que viveu em Bagdá no século IX e escreveu obras que levaram à Europa o sistema de numeração com algarismos indo-arábicos e os métodos de cálculo com ele. O título de um de seus livros também deu origem à palavra "álgebra".

Algoritmos existem muito antes dos computadores. Uma receita de bolo, as instruções para montar um móvel e o caminho que um aplicativo de mapas indica são algoritmos. Veja um, escrito em português comum:

```
algoritmo "Trocar uma lâmpada queimada"
   1. Desligar o interruptor
   2. Pegar uma escada e posicioná-la embaixo da lâmpada
   3. Subir na escada
   4. Retirar a lâmpada queimada
   5. Colocar a lâmpada nova
   6. Descer da escada
   7. Ligar o interruptor
   8. Se a lâmpada não acender, voltar ao passo 1 e verificar
      se a lâmpada nova está bem encaixada
```

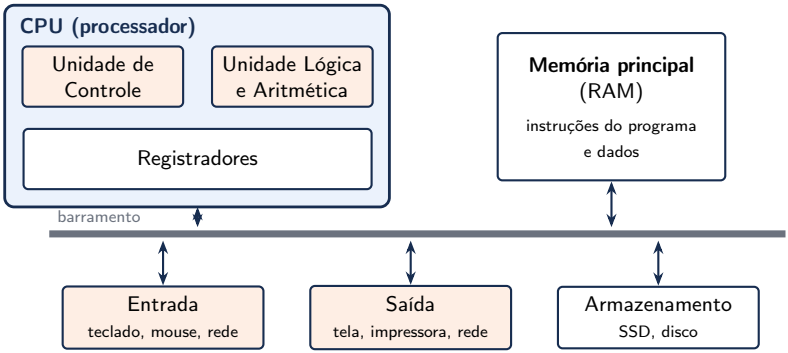
Para que uma sequência de passos seja de fato um algoritmo, ela precisa ter cinco características:

Característica	Significado
finitude	termina depois de um número finito de passos
definição	cada passo é preciso e não deixa dúvida sobre o que fazer
entrada	recebe zero ou mais dados para trabalhar
saída	produz pelo menos um resultado
efetividade	cada passo é simples o bastante para ser executado de fato

A **definição** é a mais difícil de garantir. "Coloque sal a gosto" funciona em uma receita, porque quem cozinha decide, mas um computador não tem gosto. "Aguarde a água ferver" é mais preciso, e "aguarde 5 minutos" é ainda mais. Escrever algoritmos é, em boa parte, aprender a eliminar esse tipo de ambiguidade.

1.3 Como o computador executa um algoritmo

Antes de programar, vale entender o que está sendo programado. Praticamente todos os computadores atuais, do celular ao servidor, seguem a **arquitetura de Von Neumann**, descrita pelo matemático John von Neumann em 1945, no relatório sobre o projeto do computador EDVAC. A grande ideia foi guardar o **programa na mesma memória que os dados**: para trocar o que o computador faz, basta carregar outro programa, sem mexer nos circuitos.



o programa e os dados ficam juntos na memória; a CPU busca uma instrução por vez, decodifica e executa, e repete esse ciclo bilhões de vezes por segundo

- A **CPU** (unidade central de processamento, ou processador) executa as instruções. A **unidade de controle** busca cada instrução e coordena a execução; a **unidade lógica e aritmética** (ULA) faz as contas e as comparações; os **registradores** são pequenas memórias dentro da CPU, extremamente rápidas, que guardam os valores em uso naquele instante.
- A **memória principal** (RAM) guarda o programa em execução e seus dados. É rápida, mas perde tudo quando o computador é desligado.
- O **armazenamento** (SSD ou disco) guarda arquivos e programas de forma permanente, mas é bem mais lento que a RAM. Para executar um programa, o sistema o copia do armazenamento para a memória.
- Os dispositivos de **entrada** e de **saída** trocam informações com o mundo: teclado, tela, rede.
- O **barramento** é o conjunto de ligações por onde os dados circulam entre essas partes.

A CPU trabalha em um ciclo que se repete sem parar: **busca** a próxima instrução na memória, **decodifica** o que ela pede e a **executa**. Um processador atual repete esse ciclo bilhões de vezes por segundo. Cada instrução, porém, é muito simples, como somar dois números, comparar dois valores ou copiar um dado de um lugar para outro. Tudo o que um programa faz, de uma planilha a um jogo, é uma sequência enorme dessas operações simples.

Dentro do computador, tudo é representado em **binário**, com apenas dois símbolos, 0 e 1. Cada um desses dígitos é um **bit**, e um grupo de 8 bits forma um **byte**, a unidade usada para medir memória e arquivos. Um byte pode representar $2^8 = 256$ valores diferentes, o suficiente para um número de 0 a 255 ou para uma letra.

Essa visão explica a sequência que todo programa segue, e que vai aparecer em todos os algoritmos desta apostila:

$$\textbf{entrada} \rightarrow \textbf{processamento} \rightarrow \textbf{saída}$$

Os dados chegam pela entrada, são guardados na memória, transformados pela CPU e o resultado é enviado para a saída.

1.4 Do problema ao algoritmo

O erro mais comum de quem começa a programar é sair escrevendo código antes de entender o problema. Um roteiro simples evita isso:

1. **Entender o problema**: o que precisa ser feito? Qual resultado se espera? Se possível, resolva um exemplo à mão.
2. **Identificar as entradas**: quais dados o algoritmo precisa receber?
3. **Identificar as saídas**: quais resultados ele deve produzir?
4. **Definir o processamento**: quais cálculos e decisões transformam as entradas nas saídas?
5. **Escrever o algoritmo**, em fluxograma ou em Portugol.
6. **Testar à mão**, com alguns valores, antes de passar para o computador.

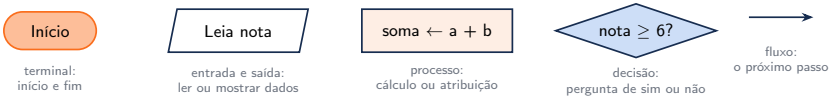
Veja o roteiro aplicado a um problema do dia a dia: um grupo de amigos quer dividir a conta de um restaurante, que cobra 10% de taxa de serviço.

Etapa	Resposta
entender	somar 10% ao valor da conta e dividir o total entre as pessoas; com uma conta de R$ 200 e 4 pessoas, cada uma paga R$ 55
entradas	o valor da conta e o número de pessoas
saída	o valor que cada pessoa paga
processamento	total ← conta × 1,10; porPessoa ← total / pessoas

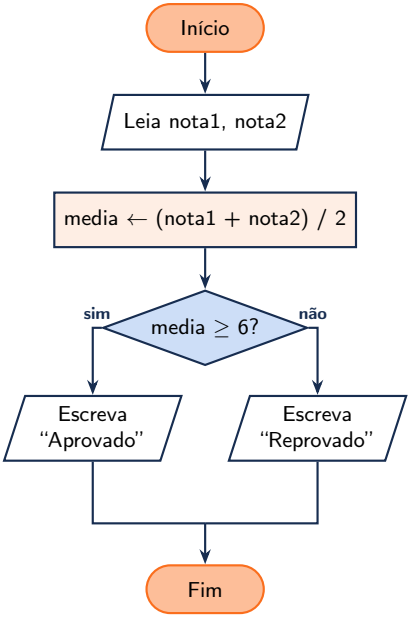
Repare que o exemplo resolvido à mão, na primeira linha, já serve de teste: quando o algoritmo estiver pronto, ele precisa dar R$ 55 para essa entrada.

1.5 Fluxogramas

O **fluxograma** representa um algoritmo com desenhos: cada tipo de passo tem uma forma geométrica, e setas mostram a ordem. Ele é útil para enxergar o caminho do algoritmo, principalmente quando há decisões, antes de se preocupar com a escrita.



O fluxograma abaixo lê duas notas, calcula a média e informa se o aluno foi aprovado. A seta ← indica **atribuição**: "media ← (nota1 + nota2) / 2" significa "calcule (nota1 + nota2) / 2 e guarde o resultado em media". O losango tem duas saídas, e o caminho seguido depende da resposta à pergunta.



Alguns cuidados deixam o fluxograma fácil de ler: ele tem um único início e um único fim; o fluxo corre de cima para baixo; toda decisão tem as duas saídas identificadas; e as setas não se cruzam. Para algoritmos grandes, porém, o desenho fica difícil de manter, e é aí que o pseudocódigo se torna mais prático.

1.6 Portugol

O **Portugol** é um **pseudocódigo** em português: uma forma estruturada de escrever algoritmos, com poucas palavras e regras fixas, mas sem os detalhes de uma linguagem de programação real. Ele permite pensar na lógica sem se preocupar com ponto e vírgula, bibliotecas ou mensagens do compilador.

Não existe um Portugol único: cada livro e cada ferramenta usa uma variação. Nesta apostila usamos a do **VisuAlg**, um programa gratuito, muito usado no ensino no Brasil, que executa algoritmos em Portugol. Assim, todo algoritmo das próximas páginas pode ser digitado e testado nele. Outra ferramenta conhecida é o Portugol Studio, que usa uma sintaxe um pouco diferente, com chaves.

Este é o algoritmo do fluxograma anterior em Portugol:

```
algoritmo "Media"
var
   nota1, nota2, media: real
inicio
   escreva("Digite a nota 1: ")
   leia(nota1)
```

```
   escreva("Digite a nota 2: ")
   leia(nota2)
   media <- (nota1 + nota2) / 2
   se media >= 6 entao
      escreval("Aprovado! Média: ", media)
   senao
      escreval("Reprovado. Média: ", media)
   fimse
fimalgoritmo
```

- `algoritmo "Media"` dá um nome ao algoritmo.
- Em `var` são declaradas as **variáveis**, os espaços da memória onde os dados ficam guardados, cada uma com seu **tipo**: `inteiro`, `real` (com casas decimais), `caractere` (texto) ou `logico` (verdadeiro ou falso). O Capítulo 3 trata disso em detalhe.
- Entre `inicio` e `fimalgoritmo` ficam os comandos, executados de cima para baixo.
- `leia` recebe um valor digitado; `escreva` mostra um texto ou valor na tela, e `escreval` faz o mesmo e pula para a linha seguinte.
- `<-` é a atribuição, a mesma seta do fluxograma.
- `se ... entao ... senao ... fimse` escolhe entre dois caminhos, como o losango.

O recuo das linhas internas não é obrigatório, mas mostra visualmente o que está dentro de quê. Acostume-se a usá-lo desde já: em C++ ele vai ser igualmente importante para a leitura.

O algoritmo da conta do restaurante, da Seção 1.4, fica assim:

```
algoritmo "DividirConta"
var
   conta, total, porPessoa: real
   pessoas: inteiro
inicio
   escreva("Valor da conta: ")
   leia(conta)
   escreva("Número de pessoas: ")
   leia(pessoas)
   total <- conta * 1.10            // 10% de serviço
   porPessoa <- total / pessoas
   escreval("Cada pessoa paga R$ ", porPessoa)
fimalgoritmo
```

O que vem depois de `//` é um **comentário**: uma anotação para quem lê, que não é executada. Note também que, no Portugol e nas linguagens de programação, o separador decimal é o **ponto**, e não a vírgula: `1.10`, e não `1,10`.

Teste de mesa

O **teste de mesa** é a forma de testar um algoritmo sem computador: você executa os passos no papel, anotando em uma tabela o valor de cada variável a cada linha. Ele

revela erros de lógica antes mesmo de o programa existir. Para o algoritmo da média, vale testar um caso de cada resultado e o valor exatamente na fronteira, 6:

nota1	nota2	media	Saída
7.0	4.0	5.5	Reprovado
8.0	9.0	8.5	Aprovado
6.0	6.0	6.0	Aprovado (a condição é >=)

Se o algoritmo tivesse sido escrito com > no lugar de >=, a terceira linha mostraria o erro: um aluno com média exatamente 6 seria reprovado.

1.7 Do Portugol ao C++

O Portugol ajuda a pensar, mas o computador não o entende diretamente. Para que o algoritmo vire um programa, ele precisa ser escrito em uma **linguagem de programação**. Nesta disciplina usamos o **C++**, uma das linguagens mais usadas no mundo, presente em sistemas operacionais, jogos, navegadores e bancos de dados. Veja o algoritmo da média em C++, lado a lado com o que você já conhece:

```
#include <iostream>
using namespace std;

int main() {
    double nota1, nota2, media;       // var ... : real
    cout << "Digite a nota 1: ";      // escreva
    cin >> nota1;                     // leia
    cout << "Digite a nota 2: ";
    cin >> nota2;
    media = (nota1 + nota2) / 2;      // media <- ...
    if (media >= 6) {                 // se ... entao
        cout << "Aprovado! Média: " << media << endl;
    } else {                          // senao
        cout << "Reprovado. Média: " << media << endl;
    }                                 // fimse
    return 0;
}
```

A estrutura é a mesma; mudam as palavras e alguns sinais. As principais correspondências:

Portugol	C++
`inicio ... fimalgoritmo`	`int main() { ... }`
`real`, `inteiro`	`double`, `int`
`leia(x)`	`cin >> x;`
`escreva(x)`	`cout << x;`
`x <- 5`	`x = 5;`
`se ... entao ... fimse`	`if (...) { ... }`

Não se preocupe ainda com o `#include`, o `using namespace std` e o ponto e vírgula no fim das linhas: o Capítulo 2 explica cada parte de um programa em C++, como ele é

transformado em algo que o computador executa e como montar o seu ambiente para programar.

Erros Comuns

- **Começar pelo código.** Sem entender o problema e identificar entradas e saídas, o programa resolve o problema errado. Siga o roteiro da Seção 1.4.
- **Escrever passos ambíguos.** "Calcule o desconto" não diz qual desconto nem sobre qual valor. Cada passo precisa ser executável sem que ninguém precise adivinhar nada.
- **Criar um algoritmo que nunca termina.** "Repita até a água ferver" sem um fogão ligado não acaba. Toda repetição precisa de uma condição que um dia se torne falsa.
- **Confundir E com OU.** "Nota maior ou igual a 6 **ou** frequência maior ou igual a 75%" aprova quem só cumpriu uma das exigências.
- **Esquecer um dos caminhos da decisão.** Todo losango tem duas saídas. Pergunte sempre: e se a resposta for não?
- **Confundir atribuição com igualdade.** `media <- 6` guarda 6 em `media`; não é uma pergunta. A comparação "media é igual a 6?" é escrita de outra forma, como você verá no Capítulo 3.
- **Usar vírgula como separador decimal.** No Portugol e no C++, escreve-se `1.10`.
- **Testar só o caso fácil.** Inclua no teste de mesa os valores de fronteira, como a média exatamente 6.

Capítulo 2 – Linguagens de programação

No Capítulo 1, o algoritmo da média foi escrito em Portugol e depois em C++. O Portugol serve para pensar; o C++ é uma **linguagem de programação**, que um computador consegue de fato executar. Neste capítulo você vai entender o que é uma linguagem de programação, a diferença entre as regras de escrita de uma linguagem (sintaxe) e o significado do que se escreve (semântica), como um programa em C++ vira algo que o processador executa e como montar o ambiente para programar. No final, vai escrever programas completos com entrada e saída de dados e aprender a ler as mensagens de erro do compilador.

2.1 O que é uma linguagem de programação

O processador só entende **linguagem de máquina**: sequências de bits que representam instruções muito simples, como "some o conteúdo destes dois registradores". Escrever programas assim é possível, mas extremamente trabalhoso e cheio de erros. Por isso, ao longo da história foram criadas linguagens cada vez mais próximas da forma como as pessoas pensam:

Nível	Como é	Exemplo
linguagem de máquina	bits que o processador executa diretamente	`10110000 01100001`
montagem (*assembly*)	cada instrução de máquina ganha um nome curto	`mov al, 97`
alto nível	comandos próximos da linguagem humana e da matemática	`x = a + 97;`

Uma **linguagem de programação de alto nível** é um conjunto de palavras, símbolos e regras para escrever instruções de forma precisa, que depois são traduzidas para linguagem de máquina. Algumas das mais importantes:

- **FORTRAN** (1957): uma das primeiras linguagens de alto nível, criada para cálculos científicos.
- **C** (1972): criada por Dennis Ritchie para escrever o sistema operacional Unix. Combina comandos de alto nível com acesso direto à memória, e até hoje é a base de sistemas operacionais e de equipamentos eletrônicos.
- **C++** (1985): criada por Bjarne Stroustrup como uma extensão do C, acrescentando, entre outras coisas, a orientação a objetos (Capítulo 12). É usada em jogos, navegadores, bancos de dados e em qualquer sistema em que o desempenho importa.
- **Python** (1991) e **Java** (1995): linguagens de alto nível muito populares; o Java é estudado no Capítulo 12.

Todas as linguagens têm os mesmos ingredientes básicos: variáveis, operadores, decisões, repetições e funções, os assuntos dos próximos capítulos. Quem aprende bem a lógica e uma linguagem aprende as outras com muito mais facilidade.

2.2 Sintaxe e semântica

Assim como o português, toda linguagem de programação tem dois níveis de regras:

- A **sintaxe** define **como** escrever: quais palavras existem, em que ordem podem aparecer, onde vão os parênteses e o ponto e vírgula. "Rato o gato comeu o" tem erro de sintaxe em português: as palavras existem, mas a ordem não forma uma frase.
- A **semântica** define **o que** cada construção significa. "A pedra almoçou o professor" tem a sintaxe perfeita, mas o significado não faz sentido.

Em um programa, erros de sintaxe são encontrados pelo compilador, que se recusa a continuar. Erros de semântica são mais traiçoeiros: o programa compila e roda, mas faz algo diferente do que o programador queria. Veja um exemplo com a sintaxe correta e uma surpresa semântica:

```
#include <iostream>
using namespace std;

int main() {
    int a = 7, b = 2;
    cout << a / b << endl;      // 3, e não 3.5
    cout << a / 2.0 << endl;    // 3.5
    return 0;
}
```

Em C++, a divisão entre dois números inteiros descarta a parte decimal: `7 / 2` vale 3. Basta um dos números ter casas decimais, como `2.0`, para o resultado ser 3.5. Nada disso é um erro para o compilador: é o significado definido pela linguagem. Conhecer a semântica é o que evita esse tipo de surpresa, e o Capítulo 3 volta a esse assunto.

As peças de um programa

Para o compilador, o texto de um programa é formado por pequenas peças, chamadas **elementos léxicos** (ou *tokens*). Reconhecer cada uma ajuda a ler qualquer código:

Peça	O que é	Exemplos
palavras reservadas	têm significado fixo na linguagem	`int`, `if`, `return`
identificadores	nomes escolhidos pelo programador	`media`, `nota1`
literais	valores escritos diretamente	`7`, `2.0`, `"Olá"`
operadores	indicam operações	`+`, `=`, `>=`
pontuação	organiza a estrutura	`;`, `{ }`, `( )`

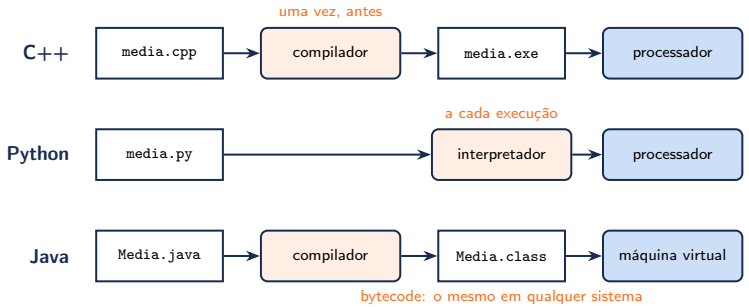
Um identificador pode ter letras, dígitos e o sublinhado (`_`), mas não pode começar com dígito nem ser uma palavra reservada. O C++ diferencia maiúsculas de minúsculas: `media`, `Media` e `MEDIA` são três nomes diferentes. Evite acentos nos identificadores e prefira nomes que digam o que a variável guarda: `precoTotal` é muito melhor que `x`.

2.3 Compilação e interpretação

Um programa escrito em uma linguagem de alto nível é chamado de **código-fonte**. Para ser executado, ele precisa ser traduzido para linguagem de máquina, e existem duas estratégias principais:

- Um **compilador** traduz o programa inteiro **uma vez**, antes de executá-lo, e gera um arquivo executável. Depois disso, o programa roda sem precisar do compilador. É o caso do C e do C++.
- Um **interpretador** lê o código-fonte e o executa aos poucos, **a cada execução**. É o caso do Python e do JavaScript.

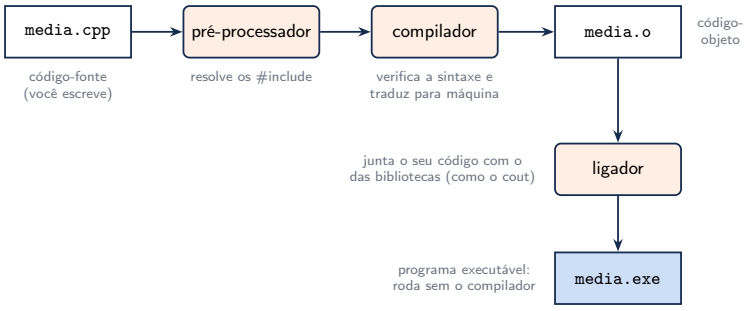
O Java usa um caminho intermediário: o compilador gera um código para uma máquina imaginária, o *bytecode*, que é executado por uma **máquina virtual** instalada no computador. Assim, o mesmo programa compilado roda em qualquer sistema que tenha a máquina virtual.



Programas compilados costumam ser mais rápidos, porque toda a tradução foi feita antes, e o compilador encontra muitos erros sem que o programa precise rodar. Programas interpretados são mais práticos para testar ideias rapidamente, mas um erro só aparece quando a linha com problema é executada.

As etapas da compilação em C++

O que chamamos de "compilar" um programa em C++ são, na verdade, três etapas, executadas em sequência por um único comando:



1. O **pré-processador** trata as linhas que começam com `#`, como o `#include`, que copia para o programa as declarações de uma biblioteca.

2. O **compilador** verifica a sintaxe e traduz o código para linguagem de máquina, gerando o **código-objeto**.
3. O **ligador** (*linker*) junta o código-objeto com o código das bibliotecas usadas, como a que implementa o `cout`, e gera o **executável**.

2.4 O ambiente de programação

Para programar em C++, você precisa de duas coisas: um **editor**, onde escreve o código, e um **compilador**. O compilador mais usado é o **g++**, do projeto GCC, gratuito e disponível para Windows, Linux e macOS. No Windows, ele é instalado por pacotes como o MinGW-w64.

Uma **IDE** (ambiente de desenvolvimento integrado) reúne editor, compilador e depurador em um só programa, e permite compilar e executar com um clique. Algumas opções gratuitas: VS Code (com a extensão de C++), Code::Blocks e Dev-C++. Também existem compiladores on-line, que funcionam no navegador sem instalar nada, úteis para os primeiros exercícios.

Mesmo usando uma IDE, vale saber compilar pelo terminal, porque é isso que ela faz por baixo:

```
$ g++ -Wall -Wextra media.cpp -o media
$ ./media
Digite a nota 1: 7
Digite a nota 2: 8
Aprovado! Média: 7.5
```

O `$` representa o terminal, e não deve ser digitado. A primeira linha compila `media.cpp` e gera o executável `media` (no Windows, `media.exe`). As opções `-Wall -Wextra` ligam os **avisos** do compilador, que apontam trechos suspeitos mesmo quando não são erros de sintaxe: use-as sempre. A segunda linha executa o programa.

2.5 A estrutura de um programa em C++

Este é o programa mais simples que faz alguma coisa visível, e cada linha dele vai aparecer em todos os programas da apostila:

```
// Meu primeiro programa em C++
#include <iostream>      // biblioteca de entrada e saída
using namespace std;     // cout em vez de std::cout

int main() {             // o programa começa aqui
    cout << "Olá, mundo!" << endl;
    return 0;            // 0 indica que terminou sem erro
}
```

- `//` inicia um **comentário**, que vai até o fim da linha e é ignorado pelo compilador. Comentários de várias linhas ficam entre `/*` e `*/`.
- `#include <iostream>` inclui a biblioteca de entrada e saída, que traz o `cout` e o `cin`.

- `using namespace std;` evita ter de escrever `std::` antes de `cout`, `cin` e `endl`, que pertencem ao "espaço de nomes" padrão, o `std`.
- `int main()` é a **função principal**: todo programa em C++ começa a ser executado por ela. O que está entre as chaves `{ }` é o **corpo** da função.
- Cada **instrução** termina com ponto e vírgula. Ele é para o C++ o que o ponto final é para uma frase.
- `return 0;` encerra o `main` e informa ao sistema que o programa terminou normalmente.

Espaços, linhas em branco e recuos não mudam o significado do programa, mas fazem enorme diferença para quem lê. O padrão desta apostila é recuar quatro espaços tudo o que está dentro de um par de chaves.

2.6 Saída de dados com cout

O `cout` (*console output*) mostra dados na tela. O operador `<<` envia cada valor para ele, e vários valores podem ser encadeados em uma mesma instrução. O `endl` termina a linha; o caractere especial `\n` faz o mesmo dentro de um texto.

```
#include <iostream>
#include <iomanip>     // fixed e setprecision
using namespace std;

int main() {
    cout << "Linha 1" << endl;
    cout << "Linha 2\n";                  // \n também quebra
    cout << "Nome:\tAna\n";               // \t: tabulação
    cout << "Ela disse \"oi\"" << endl;   // \" : aspas no texto
    cout << "C:\\Arquivos" << endl;       // \\ : a própria barra
    cout << 7 << " + " << 3 << " = " << 7 + 3 << endl;
    // 7 + 3 = 10

    double preco = 55;
    cout << preco << endl;                           // 55
    cout << fixed << setprecision(2) << preco << endl;   // 55.00
    cout << 10.0 / 3 << endl;                        // 3.33
    return 0;
}
```

As combinações que começam com a barra invertida são as **sequências de escape**: elas representam caracteres que não daria para escrever diretamente no meio do texto. Para valores com casas decimais, `fixed` junto com `setprecision(2)`, da biblioteca `<iomanip>`, mostra sempre duas casas, o formato certo para dinheiro. Essa configuração continua valendo para todos os números mostrados depois dela, como o `3.33` da última linha.

No Portugol do VisuAlg, a mesma formatação é feita com dois números depois do valor: `escreval(preco:6:2)` mostra o preço com 6 posições no total e 2 casas decimais.

2.7 Entrada de dados com cin

O `cin` (*console input*) lê dados digitados no teclado. O operador `>>` pega o próximo valor e o guarda na variável, convertendo-o para o tipo dela. Vários valores podem ser lidos de uma vez, separados por espaços ou por Enter.

O `>>` para de ler no primeiro espaço, então não serve para textos com espaços, como um nome completo. Para esses, use `getline`, que lê a linha inteira. Antes dele, `cin >> ws` descarta o Enter que o `>>` anterior deixou para trás; sem isso, o `getline` leria uma linha vazia.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int idade;
    double altura;
    cout << "Idade e altura: ";
    cin >> idade >> altura;        // ex.: 19 1.72
    string nome;
    cout << "Nome completo: ";
    cin >> ws;                     // descarta o Enter que sobrou
    getline(cin, nome);            // ex.: Ana Lima
    cout << nome << ", " << idade << " anos, " << altura << " m"
         << endl;                  // Ana Lima, 19 anos, 1.72 m
    return 0;
}
```

A variável `nome` é do tipo `string`, que guarda textos e precisa da biblioteca `<string>`. Os tipos de dados são o assunto do Capítulo 3, e os textos, do Capítulo 8.

2.8 Um programa completo

Com entrada, processamento e saída, já é possível transformar em programa o algoritmo da conta do restaurante, do Capítulo 1. Compare com o Portugol:

```
algoritmo "DividirConta"
var
   conta, total, porPessoa: real
   pessoas: inteiro
inicio
   escreva("Valor da conta: ")
   leia(conta)
   escreva("Número de pessoas: ")
   leia(pessoas)
   total <- conta * 1.10
   porPessoa <- total / pessoas
   escreval("Total com serviço: R$ ", total:8:2)
   escreval("Cada pessoa paga: R$ ", porPessoa:8:2)
fimalgoritmo
```

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double conta, total, porPessoa;
    int pessoas;
    cout << "Valor da conta: ";
    cin >> conta;
    cout << "Número de pessoas: ";
    cin >> pessoas;
    total = conta * 1.10;               // 10% de serviço
    porPessoa = total / pessoas;
    cout << fixed << setprecision(2);
    cout << "Total com serviço: R$ " << total << endl;
    cout << "Cada pessoa paga: R$ " << porPessoa << endl;
    return 0;
}
// Exemplo de execução:
// Valor da conta: 200
// Número de pessoas: 4
// Total com serviço: R$ 220.00
// Cada pessoa paga: R$ 55.00
```

O resultado confere com o exemplo resolvido à mão no Capítulo 1: R$ 55 por pessoa. Repare que `total / pessoas` divide um `double` por um `int`; como um dos lados tem casas decimais, o resultado também tem, e não há a surpresa da divisão inteira da Seção 2.2.

2.9 Lendo as mensagens de erro

Errar faz parte de programar, e o compilador é um aliado: ele aponta onde está o problema. Considere este programa, em que falta o ponto e vírgula no fim do `cout`:

```
// erro de compilação: falta o ; no fim do cout
#include <iostream>
using namespace std;

int main() {
    cout << "Ola" << endl
    return 0;
}
```

O g++ responde:

```
erro.cpp: In function 'int main()':
erro.cpp:6:26: error: expected ';' before 'return'
    6 |     cout << "Ola" << endl
      |                          ^
      |                          ;
```

A mensagem diz o arquivo (`erro.cpp`), a linha (6) e a coluna (26) do problema, o que ele esperava ("expected `';'`", um ponto e vírgula) e ainda desenha onde ele deveria estar. Três dicas valem para qualquer mensagem:

- **Comece pelo primeiro erro.** Um único erro pode causar uma cascata de mensagens seguintes; corrigido o primeiro, muitas delas desaparecem.
- **Olhe também a linha anterior.** O compilador percebe a falta do ponto e vírgula só quando encontra o `return`, na linha seguinte.
- **Leia a sugestão.** Ao usar uma variável `idad` que não existe, o g++ responde `'idad' was not declared in this scope; did you mean 'idade'?`: "`idad` não foi declarada; você quis dizer `idade`?".

Além dos **erros**, que impedem a compilação, o compilador mostra **avisos** (*warnings*), que não impedem, mas indicam algo suspeito, como uma variável declarada e nunca usada. Trate todo aviso como um erro a corrigir.

Os erros que o compilador encontra são os de **compilação**. Há ainda os erros de **execução**, quando o programa compila mas para no meio, por exemplo ao dividir um inteiro por zero, e os erros de **lógica**, quando ele roda até o fim mas dá a resposta errada. Esses dois últimos só aparecem testando, e é por isso que o teste de mesa do Capítulo 1 continua valendo depois que o programa existe.

Erros Comuns

- **Esquecer o ponto e vírgula.** Toda instrução termina com `;`. Repare que a mensagem de erro costuma apontar para a linha seguinte.
- **Esquecer o `#include`.** Sem `<iostream>`, o `cout` não existe; sem `<string>` ou `<iomanip>`, `string` e `setprecision` também não.
- **Trocar maiúsculas por minúsculas.** `Cout`, `Main` e `Int` não existem. O C++ diferencia as duas.
- **Inverter os operadores do cin e do cout.** Saída usa `<<` (os dados vão para o `cout`); entrada usa `>>` (os dados vêm do `cin` para a variável).
- **Ler um texto com espaços usando `>>`.** Só a primeira palavra é lida. Use `getline`, precedido de `cin >> ws` se houver uma leitura com `>>` antes.
- **Dividir inteiros esperando casas decimais.** `7 / 2` vale 3. Use pelo menos um valor com ponto decimal, como `7 / 2.0`.
- **Usar vírgula decimal.** Em C++, assim como no Portugol, `1,10` não é um número: escreva `1.10`.
- **Ignorar os avisos.** Compile sempre com `-Wall -Wextra` e leia o que o compilador diz.

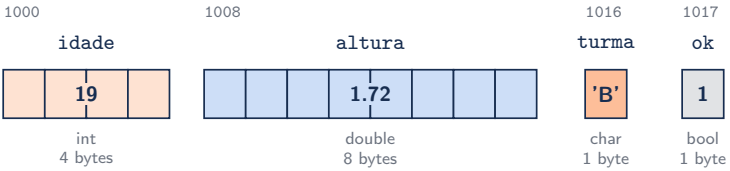
Capítulo 3 – Variáveis, tipos e operadores

Todo programa trabalha com dados: notas, preços, nomes, datas, respostas de sim ou não. Neste capítulo você vai aprender como esses dados ficam guardados na memória, em **variáveis**; por que cada variável tem um **tipo**, que define o que ela pode guardar e quanto espaço ocupa; e como combiná-los em cálculos e comparações com os **operadores**. Esses três conceitos são a matéria-prima de todos os capítulos seguintes, e boa parte dos erros de quem começa a programar vem de detalhes que aparecem aqui, como a divisão entre inteiros e a diferença entre `=` e `==`.

3.1 Variáveis

Uma **variável** é um espaço da memória, identificado por um nome, que guarda um valor. Ela tem quatro características:

- um **nome**, escolhido pelo programador, usado para se referir a ela no código;
- um **tipo**, que diz que espécie de valor ela guarda e quantos bytes ocupa;
- um **valor**, que pode mudar durante a execução (daí o nome "variável");
- um **endereço**, a posição da memória onde ela está, escolhida pelo compilador.



cada quadrado é um byte; o número acima de cada variável é o endereço do seu primeiro byte
(os endereços são ilustrativos: quem os escolhe é o compilador)

Em C++, toda variável precisa ser **declarada** antes de ser usada, indicando o tipo e o nome. O ideal é já dar um valor inicial na declaração, o que se chama **inicializar**. Uma variável declarada sem valor contém o que estiver naquele pedaço da memória, um valor imprevisível conhecido como "lixo". O g++ avisa quando isso acontece: `'x' is used uninitialized`.

Quando um valor não deve mudar nunca, como o número π ou a média mínima para aprovação, ele é declarado como **constante**, com a palavra `const`. Qualquer tentativa de alterá-lo vira um erro de compilação. Por convenção, nomes de constantes são escritos em maiúsculas.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int idade = 19;               // número inteiro
    double altura = 1.72;         // número real
    char turma = 'B';             // um caractere: aspas simples
    bool matriculado = true;      // verdadeiro ou falso
    string nome = "Ana Lima";     // texto: aspas duplas
```

```
    const double MEDIA_MINIMA = 6.0;   // constante

    idade = idade + 1;            // usa o antigo e o substitui
    cout << nome << ", " << idade << " anos, turma " << turma
         << endl;                  // Ana Lima, 20 anos, turma B
    cout << matriculado << endl;               // 1
    cout << boolalpha << matriculado << endl;   // true
    cout << "Média mínima: " << MEDIA_MINIMA << endl;   // 6
    return 0;
}
```

A linha `idade = idade + 1;` mostra como a atribuição funciona: primeiro o lado direito é calculado, usando o valor atual (19); depois o resultado (20) é guardado na variável da esquerda, substituindo o anterior. Para o `cout`, um `bool` verdadeiro aparece como 1; o `boolalpha` faz aparecer `true` e `false`.

3.2 Tipos de dados

O **tipo** de uma variável define três coisas: quais valores ela pode guardar, quanto espaço ocupa na memória e quais operações fazem sentido com ela. Os tipos básicos do C++ e seus equivalentes em Portugol são:

C++	Portugol	Guarda	Exemplos
`int`	`inteiro`	números inteiros	`19`, `-3`
`double`	`real`	números com casas decimais	`1.72`, `-0.5`
`char`	`caractere`	um único caractere	`'B'`, `'7'`
`bool`	`logico`	verdadeiro ou falso	`true`, `false`
`string`	`caractere`	um texto	`"Ana"`

Repare que o Portugol usa `caractere` tanto para uma letra quanto para um texto inteiro, enquanto o C++ separa os dois: `char` guarda exatamente um caractere, entre aspas simples, e `string` guarda um texto, entre aspas duplas. `'A'` e `"A"` são coisas diferentes em C++.

Cada tipo tem um tamanho, e o tamanho limita os valores possíveis. O tamanho exato depende do compilador e do sistema, mas nos computadores atuais costuma ser o que o programa abaixo mostra. A função `sizeof` informa quantos bytes um tipo ocupa, e a biblioteca `<climits>` traz os limites de cada tipo inteiro.

```
#include <iostream>
#include <climits>     // INT_MAX, LLONG_MAX
using namespace std;

int main() {
    cout << "int: " << sizeof(int) << " bytes, até "
         << INT_MAX << endl;       // 4 bytes, até 2147483647
    cout << "long long: " << sizeof(long long) << " bytes, até "
         << LLONG_MAX << endl;
    // 8 bytes, até 9223372036854775807
    cout << "double: " << sizeof(double) << endl;   // 8
```

```
    cout << "char: " << sizeof(char) << endl;       // 1
    cout << "bool: " << sizeof(bool) << endl;       // 1
    return 0;
}
```

Um `int` vai de cerca de −2,1 a 2,1 bilhões. É suficiente para idades, quantidades e anos, mas não para a população mundial ou para valores em centavos de uma grande empresa: para esses, use `long long`. Quando uma conta passa do limite, acontece um **estouro** (*overflow*), e o resultado fica errado sem nenhum aviso durante a execução.

O `double` guarda cerca de 15 dígitos significativos, o que é ótimo para medidas e cálculos científicos. Mas ele guarda os números em binário, e muitas frações que são exatas em decimal, como 0,1, não têm representação exata em binário. Por isso, `0.1 + 0.2` dá `0.30000000000000004`, e não exatamente `0.3`. Para a maioria das contas, essa diferença é irrelevante; ela só importa quando se compara valores reais com `==`, como você verá na Seção 3.5.

3.3 Operadores aritméticos

Os operadores aritméticos fazem as contas. Quase todos funcionam como na matemática, com duas atenções especiais: a divisão e o resto.

Operação	C++	Portugol	Exemplo em C++
soma	`+`	`+`	`17 + 5` vale 22
subtração	`-`	`-`	`17 - 5` vale 12
multiplicação	`*`	`*`	`17 * 5` vale 85
divisão	`/`	`/` ou `div`	`17 / 5` vale 3
resto	`%`	`mod` ou `%`	`17 % 5` vale 2
potência	`pow(a, b)`	`^`	`pow(2, 10)` vale 1024

Em C++, o operador `/` faz **divisão inteira** quando os dois lados são inteiros: `17 / 5` vale 3, e a parte decimal é descartada (não há arredondamento: `19 / 5` também vale 3). Se pelo menos um dos lados for real, a divisão é real: `17.0 / 5` vale 3.4. No Portugol do VisuAlg, `/` é sempre a divisão real, e a inteira se escreve `div`.

O operador `%` dá o **resto** da divisão inteira, e é muito mais útil do que parece: `n % 2` é 0 quando `n` é par; `n % 10` é o último dígito de `n`; `n % 7` diz em que dia da semana cai o dia `n` de uma contagem. Ele só funciona com inteiros.

Quando uma expressão tem vários operadores, a **precedência** decide a ordem: multiplicação, divisão e resto vêm antes de soma e subtração, e operadores do mesmo nível são calculados da esquerda para a direita. Parênteses mudam a ordem, e vale a pena usá-los sempre que deixarem a intenção mais clara.

```
#include <iostream>
using namespace std;

int main() {
    int a = 17, b = 5;
    cout << a + b << " " << a - b << " " << a * b << endl;
    // 22 12 85
```

```
    cout << a / b << endl;              // 3: divisão inteira
    cout << a % b << endl;              // 2: resto
    cout << 17.0 / 5 << endl;           // 3.4
    cout << static_cast<double>(a) / b << endl;   // 3.4
    cout << 2 + 3 * 4 << " " << (2 + 3) * 4 << endl;  // 14 20
    cout << 1234 % 10 << " " << 1234 / 10 << endl;    // 4 123

    int pontos = 10;
    pontos += 5;       // o mesmo que pontos = pontos + 5: 15
    pontos++;          // soma 1: 16
    pontos *= 2;       // o mesmo que pontos = pontos * 2: 32
    pontos--;          // subtrai 1: 31
    cout << pontos << endl;             // 31
    return 0;
}
```

O `static_cast<double>(a)` converte o valor de `a` para `double` só naquela conta, sem mudar a variável; com isso, a divisão passa a ser real. Os operadores `+=`, `-=`, `*=` e `/=` são atalhos para alterar uma variável usando o próprio valor dela, e `++` e `-` somam e subtraem 1. Eles aparecem o tempo todo nas repetições do Capítulo 5.

3.4 Conversões de tipo

Quando valores de tipos diferentes se encontram em uma conta ou atribuição, o C++ converte um deles. As conversões que não perdem informação, como de `int` para `double`, são feitas sem problema. As que perdem informação exigem cuidado:

```
#include <iostream>
#include <cmath>       // round
using namespace std;

int main() {
    double preco = 9.99;
    int reais = static_cast<int>(preco);   // 9: corta os decimais
    cout << reais << " " << round(preco) << endl;   // 9 10

    int quantidade = 7;
    double metade = quantidade / 2.0;      // 3.5
    cout << metade << endl;

    char letra = 'A';
    cout << static_cast<int>(letra) << endl;        // 65
    cout << static_cast<char>(letra + 2) << endl;   // C
    return 0;
}
```

Converter um `double` para `int` **corta** a parte decimal, sem arredondar: 9,99 vira 9. Para arredondar, use `round`, da biblioteca `<cmath>`. A conversão entre `char` e `int` mostra que, para o computador, cada caractere é um número: o código do `'A'` na tabela ASCII é 65, o do `'B'` é 66, e assim por diante. Por isso `'A' + 2` dá 67, o código do `'C'`.

3.5 Operadores relacionais

Os operadores relacionais comparam dois valores e dão como resultado um `bool`: `true` ou `false`. São eles que formam as condições das decisões e repetições dos próximos capítulos.

Pergunta	C++	Portugol	Com x = 7
igual a	`==`	`=`	`x == 7` é `true`
diferente de	`!=`	`<>`	`x != 7` é `false`
maior que	`>`	`>`	`x > 7` é `false`
menor que	`<`	`<`	`x < 10` é `true`
maior ou igual a	`>=`	`>=`	`x >= 7` é `true`
menor ou igual a	`<=`	`<=`	`x <= 6` é `false`

Atenção à igualdade: em C++, `=` é **atribuição** e `==` é **comparação**. No Portugol é o contrário, porque a atribuição é a seta `<-`, e `=` compara. Escrever `x = 7` onde se queria `x == 7` é um dos erros mais comuns em C++, e o Capítulo 4 mostra por que ele é tão traiçoeiro.

Também por causa da representação em binário, não compare valores `double` com `==`: `0.1 + 0.2 == 0.3` dá `false`. Em vez disso, verifique se a diferença entre eles é muito pequena, por exemplo menor que `0.000001`.

3.6 Operadores lógicos

Os operadores lógicos combinam condições, exatamente como os conectivos E, OU e NÃO do Capítulo 1, com a mesma tabela-verdade:

Conectivo	C++	Portugol	Verdadeiro quando
E	`&&`	`e`	as duas condições são verdadeiras
OU	`||`	`ou`	pelo menos uma é verdadeira
NÃO	`!`	`nao`	a condição é falsa

```
#include <iostream>
using namespace std;

int main() {
    double nota = 7.5;
    int frequencia = 80;              // em %
    int idade = 17;
    bool estudante = true;

    bool aprovado = nota >= 6 && frequencia >= 75;
    bool meiaEntrada = estudante || idade >= 60;
    bool menor = !(idade >= 18);
    bool notaValida = nota >= 0 && nota <= 10;

    cout << boolalpha;
    cout << aprovado << " " << meiaEntrada << endl;   // true true
```

```
    cout << menor << " " << notaValida << endl;       // true true
    cout << (0.1 + 0.2 == 0.3) << endl;               // false
    return 0;
}
```

Repare em `notaValida`: para dizer "a nota está entre 0 e 10", é preciso escrever as duas comparações ligadas por `&&`. A forma matemática `0 <= nota <= 10` compila, mas não faz o que parece: o C++ calcula `0 <= nota`, que dá `true` ou `false` (1 ou 0), e depois compara esse 1 ou 0 com 10, o que é sempre verdadeiro. O g++ avisa: `comparisons like 'X<=Y<=Z' do not have their mathematical meaning`.

Os operadores `&&` e `||` também fazem **avaliação em curto-circuito**: se a resposta já está decidida pelo lado esquerdo, o lado direito nem é calculado. Em `pessoas > 0 && total / pessoas > 50`, se `pessoas` for 0, a divisão por zero nunca acontece, porque `pessoas > 0` já é falso e o `&&` só pode dar falso.

3.7 Precedência geral

Com operadores de tipos diferentes na mesma expressão, a ordem de cálculo segue esta tabela, do primeiro para o último:

Ordem	Operadores	Tipo
1	`( )`	parênteses
2	`!`, `-` (negativo), `++`, `-`	unários
3	`*`, `/`, `%`	multiplicação, divisão e resto
4	`+`, `-`	soma e subtração
5	`<`, `<=`, `>`, `>=`	relacionais
6	`==`, `!=`	igualdade
7	`&&`	e lógico
8	`||`	ou lógico
9	`=`, `+=`, `-=`, ...	atribuição

A figura mostra a avaliação de uma expressão que mistura quase todos eles:

com nota = 7, faltas = 3 e bonus = true

```
nota + 2 * faltas >= 12 && !bonus
```

`nota + 6 >= 12 && !bonus`	1. multiplicação: 2 * 3
`13 >= 12 && !bonus`	2. soma: 7 + 6
`true && !bonus`	3. comparação: 13 >= 12
`true && false`	4. negação: !true
`false`	5. e lógico: true && false

Note que `&&` vem antes de `||`: `a || b && c` significa `a || (b && c)`. Ninguém precisa decorar a tabela inteira: na dúvida, use parênteses. Eles nunca atrapalham e deixam claro para quem lê o que se pretendia.

3.8 Funções matemáticas

A biblioteca `<cmath>` traz as funções matemáticas mais usadas:

Função	Calcula	Exemplo
`sqrt(x)`	raiz quadrada	`sqrt(16)` é 4
`pow(x, y)`	x elevado a y	`pow(2, 10)` é 1024
`abs(x)`	valor absoluto	`abs(-7)` é 7
`round(x)`	arredonda para o inteiro mais próximo	`round(2.5)` é 3
`floor(x)`	arredonda para baixo	`floor(2.7)` é 2
`ceil(x)`	arredonda para cima	`ceil(2.1)` é 3

O `ceil` resolve com facilidade problemas do tipo "quantos ônibus de 40 lugares são necessários para 130 alunos?": `ceil(130 / 40.0)` dá 4. Com a divisão inteira `130 / 40`, o resultado seria 3, e 10 alunos ficariam para trás.

3.9 Um programa completo

Um sistema de vídeos registra a duração em segundos, mas precisa mostrá-la em horas, minutos e segundos. A divisão inteira e o resto resolvem o problema: cada hora tem 3600 segundos, então `total / 3600` dá as horas completas; o que sobra, `total % 3600`, dividido por 60, dá os minutos; e `total % 60` dá os segundos restantes.

```
algoritmo "ConverterSegundos"
var
   total, horas, minutos, segundos: inteiro
inicio
   escreva("Duração em segundos: ")
   leia(total)
   horas <- total div 3600
   minutos <- (total mod 3600) div 60
   segundos <- total mod 60
   escreval(horas, "h ", minutos, "min ", segundos, "s")
fimalgoritmo
```

```
#include <iostream>
using namespace std;

int main() {
    int total;
    cout << "Duração em segundos: ";
    cin >> total;
    int horas = total / 3600;
    int minutos = (total % 3600) / 60;
    int segundos = total % 60;
    cout << horas << "h " << minutos << "min " << segundos << "s"
         << endl;
    return 0;
}
```

```
// Exemplo de execução:
// Duração em segundos: 3725
// 1h 2min 5s
```

Um teste de mesa confirma: $3725/3600 = 1$ hora; $3725 \bmod 3600 = 125$, e $125/60 = 2$ minutos; $3725 \bmod 60 = 5$ segundos. Conferindo: $3600 + 2 \times 60 + 5 = 3725$.

Erros Comuns

- **Usar uma variável sem inicializá-la.** Ela contém lixo, e o resultado muda a cada execução. Dê sempre um valor inicial.
- **Dividir inteiros esperando casas decimais.** `7 / 2` vale 3. Use `7.0 / 2` ou `static_cast<double>`.
- **Confundir `=` com `==`.** `=` atribui; `==` compara.
- **Escrever `0 <= x <= 10`.** Compila, mas é sempre verdadeiro. Use `x >= 0 && x <= 10`.
- **Comparar `double` com `==`.** `0.1 + 0.2 == 0.3` é falso. Compare a diferença com uma tolerância pequena.
- **Confundir `'A'` com `"A"`.** Aspas simples são um `char`; aspas duplas, um texto.
- **Esperar que a conversão para `int` arredonde.** Ela corta: 9,99 vira 9. Use `round`.
- **Estourar o `int`.** Contas que passam de 2,1 bilhões dão resultados errados. Use `long long` para números grandes.
- **Usar `%` com números reais.** O resto só existe para inteiros; com `double`, é erro de compilação.